INTER.act — System Documentation

What it is

A 2D browser simulation where a configurable number of robotic arms pass objects through a production pipeline. The arm count, layout, dependencies, and failure policies are all driven by a single YAML config file. Each arm runs in its own Docker container with health monitoring. If an arm goes down, downstream arms stop automatically.

Built for Review Party #1 — Ostfalia SoSe 2026, IN3 Group + AiSuNe Foundation.

Tech stack

Layer	What	Why
Backend	Flask + Flask-SocketIO	Small API, WebSocket built in
Frontend	Vanilla JS + HTML5 Canvas	Direct pixel control, no build step
Real-time	SocketIO over WebSocket	Pushes arm and object state to browser
Concurrency	Python threading + Condition variables	Arms in parallel, transfer points block without polling
Config	pipeline.yaml + pyyaml	Single file controls arm count, mode, failure policies
Containers	Docker Compose (6+ services)	Scales with arm count
Tests	pytest (28 tests)	Routes, state model, concurrent access

Dependencies: Flask 3.0.3, Flask-SocketIO 5.3.7, simple-websocket 1.1.0, requests 2.32.3, pyyaml 6.0.2, nginx

---

Config-driven architecture

Everything reads from `pipeline.yaml`. Adding an arm means adding 6 lines to this file and running `python scripts/generate_compose.py`.

```
pipeline:
  mode: dependency        # sequential | parallel | dependency
  products: 3
  speed: normal

failure_policy:
  default: isolate        # stop | skip | retry | isolate
  retry_max: 3
  retry_delay: 2.0

health:
```

```
  check_interval: 2.0
  timeout: 5.0
  max_missed: 3

arms:
  - id: 1
    label: "Arm 1"
    color: "#e74c3c"
    depends_on: []
    critical: true
  - id: 2
    depends_on: [1]
    critical: true
  # ... add more arms here
```

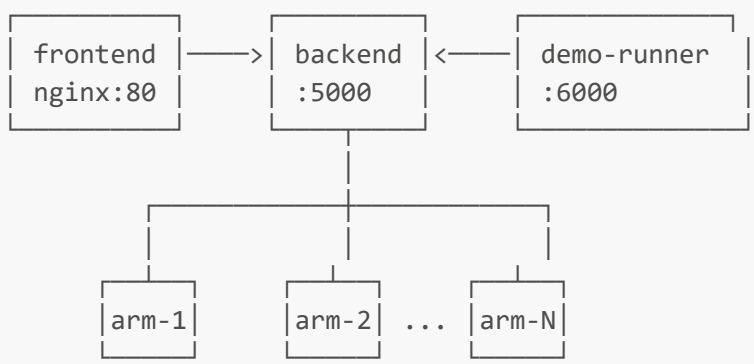
The config loader (`app/models/pipeline_config.py`) reads this file and generates:

- Scene layout (arm positions, handoff zones, intake/output zones) — all auto-computed from arm count
- Arm state dict (one entry per arm, initialized to 90 degrees)
- Valid handoff state names (`handoff_1`, `handoff_2`, ..., `handoff_{N-1}`)
- Dependency graph with cycle detection and topological ordering
- Failure policies per arm, falling back to the global default

Canvas dimensions, arm length, zone sizes, and font sizes all scale proportionally. 3 arms get full-size elements. 10 arms get compact elements. The frontend reads arm count from `/config` at page load and renders dynamically.

Architecture

6+ containers. The arm count comes from `pipeline.yaml`.



Container	Port	What it does
backend	5000	Flask API + SocketIO. Pure API — no pipeline execution. Health monitor for arm containers.
frontend	8080	nginx serving static files, proxying API and WebSocket to backend

Container	Port	What it does
demo-runner	6000	Executes the pipeline via HTTP calls to backend
arm-1..N	8001..800N	Independent microservices, one per arm

Containers find each other via Docker DNS (`http://backend:5000`, `http://arm-6:8000`).

`scripts/generate_compose.py` reads `pipeline.yaml` and writes `docker-compose.yml` with the correct number of arm services.

Why the demo-runner is a separate container

The backend used to run the pipeline in a background thread. The pipeline makes ~50 HTTP requests per demo (one per arm movement). Those requests went from Flask back to itself. Werkzeug's thread pool got saturated, and SocketIO events could not reach the browser. The arms moved on the server, but the canvas was frozen.

Moving the pipeline to its own container fixed it. The demo-runner calls the backend as an external HTTP client — same as running `python scripts/run_demo.py` from the terminal. No thread starvation because the caller is a different process.

```
Before:  Backend thread → HTTP → same Flask → thread pool full → SocketIO blocked
After:   Demo-runner → HTTP → Flask backend → normal request → SocketIO works
```

Each arm has its own brain

Each arm is an independent process. It knows its own ID (from `ARM_ID` env var) and the backend URL. It cannot accidentally control the wrong arm.

```
# CLI
python scripts/arm_1_script.py --angle 45

# Docker microservice
curl http://localhost:8001/health
curl -X POST http://localhost:8001/move -H "Content-Type: application/json" -d
'{"angle":45}'
```

Each arm microservice exposes:

- `GET /health` — liveness check
- `POST /move` — single angle command, forwarded to backend
- `POST /sequence` — multi-step motion, runs in background

Pipeline modes

Configured in `pipeline.yaml` under `pipeline.mode`:

sequential — arms chain linearly: Arm 1 → Arm 2 → ... → Arm N. N-1 transfer points. Arm order follows the config list.

parallel — each arm works independently. No transfer points, no handoffs. Each arm picks from intake and delivers to output on its own.

dependency — arms chain based on the `depends_on` field in each arm's config. For a linear chain (`depends_on: [previous_arm]`), this is identical to sequential. For branching graphs (one arm depends on two upstreams), each dependency edge gets its own transfer point.

How angles work

```
180 (LEFT)          90 (UP)           0 (RIGHT)
    <-- arm --        | arm          -- arm -->
    picks here    rest position    places here
```

Tip position: `tipX = pivot_x + length * cos(angle)`, `tipY = pivot_y - length * sin(angle)`

Arm positions are auto-computed from arm count. The spacing formula: `arm_spacing = 900 / N`. Arm length scales with spacing (max 150px). Canvas width = `spacing * (N + 1)`.

The server rejects angles outside 0-180 with HTTP 400.

Dependency awareness and failure propagation

Arms can depend on other arms via `depends_on` in the config. When an arm fails:

1. Backend health monitor pings each arm container every 2 seconds
2. Three missed pings = arm marked as dead
3. `get_all_downstream(arm_id)` finds all transitively affected arms via BFS
4. All downstream arms are marked as stopped
5. Demo-runner threads detect the change within 2 seconds (TransferPoint polls `cancel_check` every 2s)
6. Downstream arms stop themselves

Example: stopping arm 6 in a 10-arm chain marks arms 7, 8, 9, 10 as stopped. Arms 1-5 keep running.

Failure policies per arm:

- `stop` — halt and propagate to downstream (default)
- `skip` — skip the failed product, continue to next
- `retry` — retry up to `retry_max` times with `retry_delay` between attempts
- `isolate` — stop this arm but don't propagate to downstream

DAG validation runs at startup. Catches cycles and missing dependency references before anything starts.

Timing

Server side (demo-runner)

Constant	Value	What it does
MOVE_DURATION	1.5s	Base time for a 180-degree sweep
ACTION_DURATION	0.4s	Pause for pick/place/hold
fast	x0.8	Sweep 1.2s, action 0.32s
normal	x1.5	Sweep 2.25s, action 0.6s
slow	x3.0	Sweep 4.5s, action 1.2s

The speed factor multiplies every `time.sleep()` in the pipeline.

Browser side

Constant	Value	What it does
ANIM_DURATION	600ms	Arm angle easing (easeOutCubic)
OBJ_TRANSITION_MS	400ms	Object position easing between zones
Carried snap	200ms	Fast snap to arm tip before sweep

All drawing sizes (line width, pivot radius, font size, object box) scale proportionally with arm count via a scale factor `S = 3.5 / N`.

Pipeline flow

For N arms in sequential/dependency mode:

```
INTAKE -> [Arm 1] -> Handoff 1 -> [Arm 2] -> Handoff 2 -> ... -> [Arm N] -> OUTPUT
```

Object states: `intake -> carried(arm1) -> handoff_1 -> carried(arm2) -> ... -> output`

Transfer points use `threading.Condition`:

- `place(obj_id)` — deposits object, wakes downstream arm
- `wait_and_pick(cancel_check)` — blocks until object arrives, checks `cancel_check` every 2 seconds for upstream failures
- `wait_until_empty(cancel_check)` — blocks until point is cleared, also checks cancellation

How the Run Demo button works

1. User clicks "Run Demo"

2. Browser sends `POST /demo/start` to backend
3. Backend proxies to `POST http://demo-runner:6000/run`
4. Demo-runner reads `pipeline.yaml`, builds N arm controllers with N-1 transfer points
5. Pipeline calls `POST /set_angle`, `/object_event`, `/log` on the backend via HTTP
6. Backend emits SocketIO events to the browser
7. Browser animates arms and objects in real time
8. Browser polls `GET /demo/status` every second
9. When done, button changes from "Running..." to "Complete"

Health monitoring

The backend runs a background thread (`app/services/health_monitor.py`) that pings each arm container's `/health` endpoint every `check_interval` seconds (default 2s).

- 1 missed ping: arm marked as "unhealthy"
- 3 missed pings (`max_missed`): arm marked as "dead"
- Dead arm triggers `mark_arm_failed()` which cascades to all downstream arms

No API call needed to trigger this. Just `docker stop interact-arm-6-1` and the system reacts within 6 seconds.

The health monitor detects:

- Container stopped (`docker stop`)
- Container crashed (process died)
- Network failure (container unreachable)

Problems we ran into and fixed

#	What happened	Why	Fix
1	Arms collided at handoff zones	Both arm tips converge at the same pixel at 0/180 degrees	Signal fires at 0 degrees, ACTION_DURATION pause gives clearance
2	Arms vibrated at handoff zones	Exponential lerp never reaches target, rapid target changes caused oscillation	Time-based easeOutCubic, 600ms guaranteed completion
3	Objects teleported on pickup	SocketIO event arrived before browser finished the arm animation	200ms position easing bridges the network timing gap
4	Objects followed arm during retract	Object still in "carried" state while arm retracted	Object detaches before arm starts retracting
5	Transfer point lost signals	Event + Lock gap between signal receipt and lock acquisition	Single Condition variable, wait_for() is atomic

#	What happened	Why	Fix
6	CPU burned during polling	Busy-wait loop at 20Hz	Condition.wait_until_empty() blocks at OS level
7	Object state corrupted on reconnect	No lock on object state dict	Added threading.Lock
8	Arm threads crashed silently	Exceptions swallowed, pipeline hung	try/except wrapper, errors in pipeline summary
9	Arms idle between handoffs	Signal fired after retract, adding 480ms dead time	Signal fires before retract, parallel operation
10	Objects overflowed zone bounds	Fixed 24px spacing	Dynamic spacing compresses to fit
11	Canvas frozen when demo ran in backend	HTTP self-calls starved Werkzeug thread pool	Moved pipeline to demo-runner container
12	Downstream arms kept running after upstream stopped	No dependency awareness, 60s timeout before detection	Health monitor + cancel_check every 2s in TransferPoint

Remaining limitations

- Browser reconnection fetches arm state and object state in separate requests (not atomic)
- SocketIO broadcasts full state on every angle change (no batching)

API reference

Endpoint	Method	Body	What it does
/	GET	—	Simulation page
/health	GET	—	Health check
/set_angle	POST	{"arm_id": 1, "angle": 45}	Move an arm
/state	GET	—	All arm states
/state/<id>	GET	—	One arm's state
/config	GET	—	Scene layout (dynamic, from pipeline.yaml)
/object_event	POST	{"object_id": "Obj1", "state": "carried", "carrier_arm": 1}	Track object position
/objects	GET	—	All tracked objects
/objects/reset	POST	—	Clear tracked objects
/log	POST	{"source": "arm_1", "message": "..."}	Send log to browser panel

Endpoint	Method	Body	What it does
/arms/health	GET	—	All arm health states
/arms/<id>/health	POST	{"health": "alive"}	Report arm health
/arms/<id>/stop	POST	{"reason": "..."}	Stop arm + cascade to downstream
/arms/<id>/allowed	GET	—	Check if arm can execute
/pipeline/status	GET	—	Full pipeline: health, DAG validity, topo order
/reload_config	POST	—	Hot-reload pipeline.yaml without restart
/demo/start	POST	{"products": 3, "speed": "normal"}	Start demo (proxied to demo-runner)
/demo/status	GET	—	Is demo running
/demo/stop	POST	—	Stop demo

Object states: `intake`, `carried`, `handoff_1`, `handoff_2`, ..., `handoff_{N-1}`, `output`

How to run

Docker:

```
docker compose up --build
# http://localhost:5000
# Click "Run Demo"
```

Local:

```
pip install -r requirements.txt
python run.py
# http://localhost:5000
python scripts/run_demo.py --speed normal --products 3
```

Add an arm:

```
# 1. Edit pipeline.yaml
# 2. Regenerate compose:
python scripts/generate_compose.py
# 3. Rebuild:
docker compose up --build
```

Stop an arm (test failure propagation):

```
docker stop interact-arm-6-1
# Arms 7+ stop within ~6 seconds
```

Hot-reload config:

```
curl -X POST http://localhost:5000/reload_config
```

Run tests:

```
pytest tests/ -v    # 28 tests
```